

Reference-Side Contract Inference for Unmodified Scientific CLIs

Anonymous Authors
Paper under double-blind review

Abstract

A whole-command cache misses when a reference database is replaced, because the reference bytes changed. Hand-built incremental search repairs one tool’s scores in that tool’s code. This paper centres on one generic fitter. With no per-tool code, it infers for an unmodified CLI $F(\text{records}, \text{reference})$ that the output decomposes over reference entries, and which numeric columns carry a global normalizer drawn from a closed family: identity, entry count, total entry length, or per-key row count. Ties are broken by an entry-length partition. If no member fits, or a tie survives, the fitter refuses and the caller runs the tool once on the whole reference. A later release then reruns only changed entries, merges, and rescales. The prior-art verdict on that pair of inferences is narrow. iBLAST, iSeqSearch, and GeStore are hand-built per tool. KumQuat infers input-stream combiners and does not fit a normalizer. Record-side reuse with a copy-aware minimal key is secondary. Safety is infer-or-refuse. The probes are credited mechanisms, not a novelty claim. On `hmmsearch -cut_ga` the fitter ships the measured contract, and it refuses the same `argv` without that flag. Locked record-side cumulative wall speedups on `hmmsearch` are $2.740\times$ and $6.847\times$; a post-hoc correction for a stock-time fit that misses every sampled genome lowers them to $1.856\times$ and $4.404\times$. Deployed in-scope unsafe-ship is 4/224. A timed Pfam 38.1 to 38.2 re-annotation, a DIAMOND comparison, and whole-command baselines are **[PENDING: not yet measured]**.

Keywords: incremental computation, black-box inference, scientific CLIs, HMMER, reference databases

1 Introduction

Profile search and variant annotation are run again when the reference changes. Pfam 38.2 contains 30,134 models, of which 26,082 are unchanged from 38.1 under the strict entry hash, so 4,052 entries are new or changed. A cache keyed by the reference file sees a miss on every query. The unchanged models are the reusable work, if the tool’s output on

a reference is the merge of its outputs on the entries, up to a normalizer the merge can reapply.

Whole-command incrementalizers already wrap unmodified binaries. Rattle, Riker, ProcessCache, and INCR skip a command when its traced inputs are unchanged [7, 26, 28, 30]. A new Pfam release changes the reference argument, so the command reruns. Hand-built incremental search does the reference-side job for one tool family. iBLAST writes a Karlin–Altschul or Spouge repair for NCBI BLAST [8, 16]. iSeqSearch copies that rescale onto m8 producers, including DIAMOND [31]. GeStore incrementalizes unmodified genomics tools with a per-tool plugin; its BLAST plugin repairs E-values and its HMMER plugin does not [23]. KumQuat infers a combiner for an unmodified Unix command split on its input stream [25]. The combiner language has no global scale factor tied to a reference.

The contribution is the following, and nothing further.

1. **Primary: reference-side contract inference.** One generic fitter, with no per-tool code, infers for an unmodified CLI $F(\text{records}, \text{reference})$ (a) that the output decomposes over reference entries and (b) which numeric columns carry a global normalizer, and its form, from a closed family (identity, entry count, total entry length, per-key row count), with a tie-break partition and refusal. This enables re-annotation across reference releases by rerunning only changed entries. The prior-art verdict is **narrow**. The surviving claim is the sentence in the prior-art pass: no prior system infers, for an unmodified CLI and with no per-tool plugin, both that the output decomposes over reference entries and which numeric columns are global normalizers of a form fitted from a small fixed family, so that a later reference release can rerun only new or changed entries and rescale those columns. iBLAST, iSeqSearch, and GeStore are hand-built per tool. KumQuat is input-side only.
2. **Secondary: record-side reuse with a copy-aware minimal key.** Fields the output depends on split into transport fields, which are copied, and the rest. The cache key is

Table 1: Amdahl split of one stock run. Shares are $(a)/(a + bN)$ and $(bN)/(a + bN)$. The hit-all ceiling is $(a + bN)/(a + w)$, projected from the same fit. Wrapper w is the mean of three `cat`-equivalent reads.

	hmmscan	hmmsearch
a (s)	31.960	132.558
b (s/record)	0.72286	0.11890
bN (s)	3030.210	498.409
w (s)	0.01906	0.01834
a share	1.044%	21.009%
bN share	98.956%	78.991%
ceiling	95.76×	4.759×

the rest. Transport fields are spliced from the new record. On the pre-registered workloads the dependency-key recall is 0.5567 of the copy-aware recall for VCF/SnpEff and 0 at every k for GenBank proteins under hmmscan. The RefSeq control ratio is 1 at every k . The locked kill rule did not fire.

3. **Safety is infer-or-refuse.** Determinism, shuffle, subset, field perturbation, round-trip, and pre-flight decide whether a contract ships. These are metamorphic tests, a Dune-style audit, and group testing of subsets [6, 10, 24]. They are not claimed as novel.

What is measured, and what is not, is separated in the evaluation. The fitter recovers the `hmmscan -cut_ga` contract and refuses the E-value-thresholded `argv` (§6.3). Record-side `hmmsearch` savings are reported as the locked imputation and, beside it, a post-hoc correction (§6.2). The deployed probe suite’s in-scope unsafe-ship rate is 4/224 (§6.1). A timed 38.1→38.2 re-annotation, DIAMOND against iSeqSearch, head-to-head baselines, and a boundary refusal on a further unmodified CLI are [PENDING: RQ4–RQ7].

2 Background and cost model

One run of a per-record tool on n records is modelled as

$$T = a + b \cdot n. \quad (1)$$

Startup a does not shrink with n . The per-record term is b . A wrapper that parses, hashes, and splices adds w on a cached run. A one-time probe costs P . The screen that chose the workload fitted (1) on an exclusive node of a university cluster (AMD EPYC 7542) for Pfam 38.2 and $N = 4192$ proteins (Table 1).

The dominant term on this screen is $b \cdot n$. Record memoization can remove only the hit portion of that term. Even with every record a hit, `hmmscan`’s projected ceiling is 95.76× and `hmmsearch`’s is 4.759×, because `hmmsearch` pays 21.009% of a full run in startup. Optimizing w is the wrong target: the

`hmmscan` wrapper is 0.01906s against a 3030.210s record term.

Reference-side reuse has the same shape with a different miss fraction. Let c be the fraction of reference entries that are new or changed. Under the assumptions in §4, one re-annotation costs $a + cbn$ rather than $a + bn$. The probe that learns the contract is paid once. On the screen’s `hmmscan` parameters that probe is a small fraction of bn when c is the measured 38.1→38.2 churn (§4). The timed confirmation of that projection is [PENDING: RQ4].

3 Design

Figure 1 is the wrapper. Phase one probes an unmodified binary and either emits a contract or refuses. Phase two, on later inputs, runs the binary only on the work the contract says is new, then merges. Two stages are the contribution: reference-side inference, and the rescaling merge that uses the fitted member. Record-side key inference is the secondary mechanism. Safety probes are the gate.

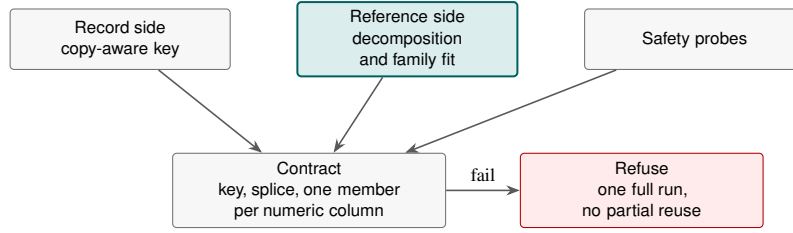
3.1 What is inferred

The tool is an unmodified function of two `argv`-named inputs, $F(\text{records}, \text{reference})$. The fitter does not read the tool’s source, a manual, or the environment. Inputs outside `argv` are fault class F6 and stay outside the guarantee (§8). A per-format parser splits the reference into entries and exposes each entry’s length. Parsers in scope are HMMER3 profile text (length is the `LENG` integer; volatile tags are `DATE`, `BM`, `SM`, `COM`), FASTA (length is the residue count), and a one-line fixture format used only in tests. Length is data on the parser. The fitter refuses if the format has no parser or a profile has no `LENG` line. Entry identity is the SHA-256 of the entry after volatile lines are dropped. An entry whose hash is stable across a release is the unit of reuse.

3.2 Record side

A record’s fields partition into a dependency set D and irrelevant fields. Inside D , transport fields U are those whose perturbation reappears at fixed output positions and changes nothing else. The copy-aware key is $K^* = D \setminus U$. The cache stores the produced fields under a hash of $x|_{K^*}$ and splices the new record’s transport fields on a hit. Joint transport—the same identity when every transport field is substituted at once—is tested on the probe sample. It is not proved for the whole domain. The minimality argument in §4 says that, when joint transport holds, K^* is a sound splice key and no smaller sound splice key exists. Dependency inference that keys all of D is prior art (§7); the secondary claim is the split that drops pure copies from the key.

1 Probe and infer, once per argv



2 Every later run

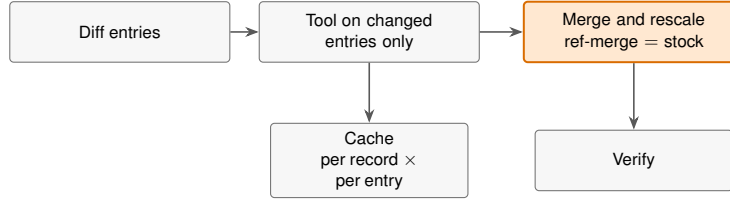


Figure 1: Two phases. Teal is reference-side inference. Orange is the rescaling merge. A refusal runs the tool once on the full inputs and does not reuse probe rows. Later output matches stock under ref-merge (§4), which is equality of hit sets and unnormalized columns, and printed-precision agreement on normalized columns.

3.3 Reference side

Figure 2 is the reference-side mechanism.

Decomposition. The primary probe draws $k = 2$ parts of the reference and a sample of $n_p = 300$ records, then invokes F on the whole reference and on each part ($k + 1 = 3$ calls, plus two more if a tie-break runs). T1 and T2 reuse the historical halves from the earlier kill tests rather than the Fisher–Yates partition pinned for new references. A row key is the format’s record key, entry key, and optional row index. The fitter ships decomposition only together with a completed normalizer assignment, and only when the union of the parts matches the whole under ref-merge: equal row-key sets, byte-identical non-numeric columns, byte-identical identity columns, and the half-ULP predicate of §4 on every linearly normalized column.

Closed family. A column is numeric when every aligned token parses as a finite float. A mixed column refuses the argv. The members, fixed before the fitter was run, are:

1. **identity:** the printed token is byte-identical on every aligned row.
2. **entry_count:** rescale by $|R|/|R_{\text{part}}|$.
3. **total_entry_length:** rescale by $L(R)/L(R_{\text{part}})$, with L the sum of parser lengths.

4. **per_key_row_count** of an output table of the same run: rescale by the ratio of row counts for that record key. A zero denominator does not fit.

No member’s definition names a tool, a flag, or a HMMER statistic. Entry count and per-key row count recover HMMER’s database size and its per-sequence reported-target count when the probe says so. A candidate fits when every aligned row satisfies the consistency predicate. Two candidates that agree with each other on every row, within that predicate, are one candidate. An all-zero column collapses to identity.

Tie-break and refusal. A primary split that balances both entry count and total length gives the same factor to both size measures, so they tie even when only one is the law. The disambiguating partition orders entries by length descending, then by content hash ascending, and fills one part until it holds at least half the total length and is a proper subset. The complement is the other part. A tied candidate survives only if it still fits. If the probe leaves zero members, or more than one distinct member, the column refuses. A column that no candidate fits refuses the whole reference-incremental path for that argv. The caller runs one full F and does not splice a partially fitted column into the output. This is stricter than the earlier column-wise rule that recomputed a single non-fitting column and reused the others. `-cut_ga` and the default threshold are different contracts.

The primary partition used for new references is a

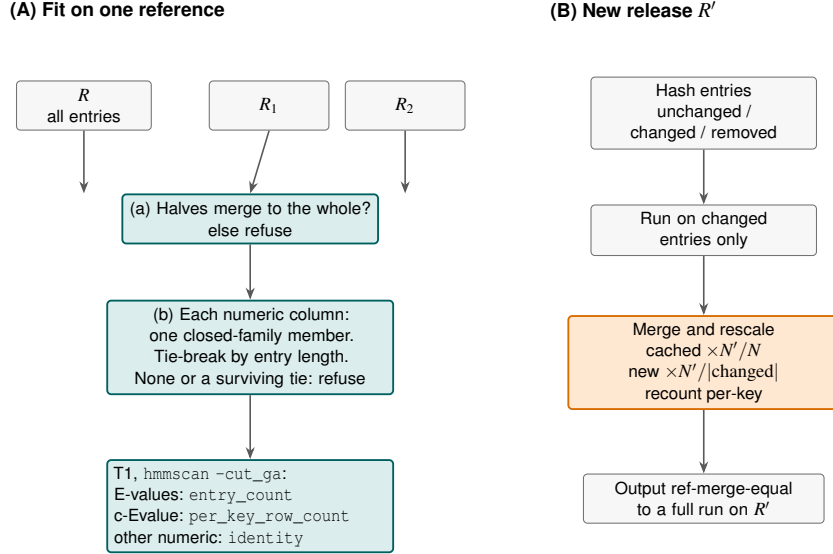


Figure 2: Reference-side inference and reuse. (A) The primary probe runs the same records on R , R_1 , and R_2 . Decomposition fails closed. Each numeric column must match one member of the closed family; a tie is broken by a length-unbalanced partition, and a column that still ties, or matches nothing, refuses the argv. The rules in the teal box are the T1 assignment for `hmmscan -cut_ga`, not code in the fitter (§6.3). (B) On a later release the entry-count member rescales cached rows by N'/N and rows from the changed sub-reference by $N'/|\text{changed}|$. Per-key counts are recounted on the merged row set. Total entry length uses sums of parser lengths in place of N . Equality to stock is ref-merge.

Fisher–Yates shuffle on `random.Random(seed).random`, not CPython’s `shuffle`, then a cut into parts whose sizes differ by at most one. The pinned check is seed 20261006 and $n = 4$: the shuffled index list is $[0, 3, 2, 1]$. An implementation that disagrees on that input is the wrong partitioner. T1 and T2 do not use it. They reuse halves already drawn by the kill-test script.

3.4 Merge and rescale

On a new reference the fitter hashes entries into unchanged, changed, and removed. Removed rows are dropped. Unchanged rows are reused. Changed and new entries are scanned as a sub-reference, after the caller-declared index command (`hmmppress` for HMMER; the command is data, not a compiled tool name). Cached numeric cells are multiplied by the fitted factor from the old reference to the new one. Cells produced on the sub-reference are multiplied by the factor from that sub-reference to the new one. For entry count those factors are N'/N and $N'/|\text{changed}|$. For total entry length they are the corresponding length ratios. Per-key row counts are computed from the merged row set after the merge, because the normalizer depends on which rows survived. The value is re-printed with the source token’s significant digits. Identity columns are copied as bytes.

3.5 Verification

Before a contract ships, the record-side probes test determinism, order, subset invariance on batches of size 1, 2, 4, 8, and field perturbation. A perturbation that changes only copied fields classifies the field as transport. A perturbation that changes produced fields widens the key. The round-trip guard rebuilds perturbed output from the inferred contract and compares it to the tool; it was added after a cached-MATCH stop, recorded as `STOP_MATCH` at collection A, `hmmsearch`, position 5. The first run on a contract must match a full run under the relation for that path (record-body or whitespace MATCH on the record side; ref-merge on the reference side). In deployment the hit audit re-executes each hit with probability $q = 0.02$ and refuses on disagreement. `-verify full` compares against a stock run. A GeStore-style baseline runs the decomposition probe and refuses any numeric column that is not identity, so a size-dependent column cannot be reused.

Record-side global dependence and reference-side normalization are different decisions. A record whose output depends on its neighbors, or on the stream length, is a refusal on the record side: there is no per-record key. A numeric column that scales with one family member is an assignment on the reference side, and the merge reapplies that scale. Table 2 is the split. The mechanisms in the left column are metamorphic tests and an audit. The right column is the contract inference.

Table 2: What a probe observation becomes. Record-side global dependence refuses. Reference-side scale, when it is a single family member, is inferred and rescaled.

Observation	Decision
Perturbation changes only copied positions	transport field; splice it
Perturbation changes produced fields	widen the key
Same argv, different output	refuse, nondeterministic
Output depends on order or neighbors	refuse, record side
Column matches one family member	assign that member
Column matches none, or a tie survives	refuse the argv
Tool reads inputs outside argv	outside the guarantee (F6)

4 Formal model

The statements below are the definitions used by the fitter and the consequences of those definitions under the assumptions written next to them. They are not claimed as theorems about HMMER beyond what the probes measure.

4.1 Decomposition and the family

Let R be a reference with $|R| \geq 2$, split into parts that are non-empty, disjoint, and exhaustive. Write $\text{keys}(F(X, R))$ for the set of row keys in the output. **Decomposition** holds for a probe sample X when, after the normalizer assignment, $\text{keys}(F(X, R))$ equals the union of $\text{keys}(F(X, R_i))$, no key appears in two parts, non-numeric and identity columns are byte-identical on aligned rows, and every linear column satisfies the consistency predicate. The fitter does not ship decomposition without an assignment.

The **normalizer family** is the four members in §3.3. It is closed: a fifth measure, including Karlin–Altschul effective length, is outside the claim unless a later addendum marked post-hoc extends the family. Results measured before that addendum would not confirm the extension. The family is not the polynomial search of Zhang et al. [32], which fits linear and quadratic relations between scalar outputs of a library function on affinely related numeric inputs.

4.2 Consistency predicate

For a printed token, let dec be the number of digits after the decimal point in the mantissa, or zero if there is no decimal point, and let exp be the integer exponent if the token contains e or E , otherwise zero.

$$\text{half-ULP}(\text{printed}) = 0.5 \times 10^{-\text{dec}} \times 10^{\text{exp}}.$$

A part value v , rescaled by a finite factor ϕ , is consistent with a whole value w when

$$|v\phi - w| \leq \text{half-ULP}(w_{\text{printed}}) + \text{half-ULP}(v_{\text{printed}}) \cdot |\phi|. \quad (2)$$

This is the predicate the kill tests and the fitter implement. Identity is the case $\phi = 1$ with byte-identical tokens.

4.3 Ref-merge error bound

The merge does not emit the real product $v\phi$. It re-prints that product at the source token’s precision. Assume that the re-printed token emitted satisfies $|\text{emitted} - v\phi| \leq \text{half-ULP}(\text{emitted})$, and that the probe accepted the row under (2) with source the cached printed value and stock the full-run printed value. The triangle inequality then gives

$$|\text{emitted} - \text{stock}| \leq \text{half-ULP}(\text{stock}) + \text{half-ULP}(\text{emitted}) + |\phi| \cdot \text{half-ULP}(\text{source}). \quad (3)$$

Each half-ULP term is the quantity defined above, evaluated on that printed token. This is the same bound as $\frac{1}{2}$ ulp of each token. The bound is a relation between printed numbers. It is not a claim about the tool’s real-valued score before printing, and it is not byte identity. Ref-merge uses byte identity for hit sets, unnormalized columns, and identity columns, and (3) for columns assigned a linear member. If the re-print assumption fails for a format, `-verify full` is the check that compares the emitted table to stock.

4.4 Break-even churn

One later re-annotation, with the probe already defined, uses

$$\begin{aligned} T_{\text{stock}}(n) &= a + bn, \\ T_{\text{inc}}(n, c) &= a + cbn, \\ S(c) &= (1 - c)bn. \end{aligned}$$

Assumptions, all stated, none of them a new measurement: a does not shrink on a smaller reference; b scales linearly with the fraction of entries searched; the two parts partition the entries, so their per-record costs sum to bn_p ; merge and rescale are ignored next to these terms. The measured `hmm-scan` wrapper is 3.60×10^{-5} of the primary probe cost below, so w is left out of P . Per-entry cost is not assumed uniform. That is why a timed release run is required before a speedup is claimed.

With $k = 2$, the primary probe costs $P = 3a + 2bn_p$. The tie-break reuses the whole-reference run and adds two calls, for $P_{\text{both}} = 5a + 3bn_p$. Savings on one later re-annotation exceed the probe when $bn > P$ and $c < c^*$, where

$$c^* = 1 - \frac{P}{bn}. \quad (4)$$

More releases only amortize P . Substituting the screen’s `hmm-scan` fit ($a = 31.960$ s, $b = 0.72286$ s/record, $n = 4192$, $n_p =$

Table 3: Projected probe cost and break-even churn for one hmmscan re-annotation at the screen’s a , b , and n . S uses c_{R1} .

Probe	P (s)	c^*	S (s)	S/P
Primary	529.592	0.8252	2622.749	4.952
Both	810.368	0.7326	2622.749	3.236

300) and the measured churn $c_{R1} = 4052/30134 = 0.13447$ gives Table 3. These cells are projected from (4) and $S(c)$. They are not timings of a 38.2 run. Length-weighted churn on the same release pair is $c_{\text{length}} = 600557/4754065 = 0.12632$, lower than the count churn because changed and new models are shorter than average. The confirmatory gate uses the count churn. The measurement is [PENDING: RQ4].

c_{R1} lies below both thresholds, so on these assumptions one re-annotation’s projected savings cover the probe, including the tie-break. The attack that the probe costs more than the savings lands if a release has $c > c^*$, or if b does not scale with the fraction of entries searched. Neither is answered by Table 3.

4.5 Copy-aware key

Assume joint transport, as defined in §3.2, and that irrelevant fields do not affect the per-record function. Then K^* is a sound splice key: the output is determined by $x|_{K^*}$ together with the transport values written at their positions. Every sound splice key contains K^* , because a field in K^* left out of the key would have to be transport or irrelevant, contradicting the partition. Recall is therefore at least as high under K^* as under any larger sound key, including the full dependency set D . The probes test single-field transport and one all-at-once substitution. A violation they miss is a residual of the same kind as the probe miss rate in §6.1, and the audit catches a hit-affecting fault with probability $1 - (1 - q)^h$ on h hits. The argument does not extend past what the probes establish.

5 Implementation

The reference-side path is a library the scripts call. $F(\text{records}, \text{reference})$ is the reference runner. The reference is the argv-named file that is not the record input and parses as entries, unless a flag names it. Per-format parsers cover profile text whose header token is `HMMS3/f`, FASTA, and the fixture lines. The strict entry hash is shared with the kill-test script. The probe parameters are the pre-registered ones. There are two parts and 300 records, seed 20261006. T1 and T2 reuse the historical shuffle. Cache rows are keyed by record key, entry content hash, and argv with the reference path removed. A later reference reuses unchanged entries, scans changed entries as a sub-reference, drops removed entries, rescales fitted columns, recomputes per-key counts from the merged

Table 4: Probe evaluation. Full MATCH is the upper bound. Audit is the deployed mode.

	Full MATCH	Audit
Unsafe-ship	33/256	68/288
In-scope	1/224	4/224
Excl. F6-env only	—	36/256
False-refuse	0/20	0/20
Audit-only catches	—	12
F6-env unsafe	32/32	32/32
F6-file unsafe	—	32/32

rows, and re-renders. Sidecar indexes are built by a caller-declared prep command. The command is data. No tool name is compiled into the fitter. Refusal is one full run, with a stated reason, when there are fewer than two entries, the record input is empty, a column is mixed or matches no member, a non-numeric column is not byte-identical, the tie-break cannot separate the survivors, primary row-key sets differ, or prep or the tool fails. A tool failure is not cached. The GeStore baseline refuses when a numeric column is not identity. Audit mismatch and full-verify mismatch are separate refusal reasons.

The record-side path is the existing VCF and FASTA inference, cache, and audit. It is unchanged by the reference fitter. Ship controls that must still hold are SnpEff re-assembly on 52,638 records (41,447 hits, 11,191 misses) and `bcftools +fill-tags` widened to `SAMPLES` with 23,072 hits on HG00099.

6 Evaluation

Each research question uses committed results or is marked pending. Anything computed after seeing a fit error is labelled post-hoc in the text. Hmmscan record-side savings were secondary before the runs. They are reported in §6.2 as POST-HOC and are not the pre-registered primary.

6.1 RQ1: Safety

The probe suite is a measurement of this wrapper, not a new testing theory. The full-MATCH run is an upper bound: the stock tool is invoked on the evaluation input. The deployed mode is probes plus the hit audit, with no stock run on that input. Canonical in-scope cells exclude F6-env and F6-file. The macOS host had no `strace`, so F6-file is unmeasured there even though probe-time tracing on Linux is the intended cover. Table 4 is taken from the two result writeups.

The one full-MATCH in-scope miss is VCF field-dependence at fault rate $p = 0.01$ and probe size 50. No live record fell in that draw, the key was not widened, and a later input with changed identifiers disagreed. That cell remains under the audit. The audit adds two FASTA field-dependence

misses and one subset miss, because full MATCH no longer runs, and it recovers 12 cells the probes missed. False refusals on the ship controls are 0/20.

If a fault hits each record independently with probability p , a probe of n records misses it with probability $(1 - p)^n$. At $p = 0.01$ and $n = 50$ that probability is 0.6050. The measured miss at that cell is consistent with the formula. It is not a guarantee of zero unsafe-ship. F6-env is undetectable by these probes and is a limitation.

6.2 RQ2: Record-side savings

The pre-registered primary is hmmsearch on two proteome collections, A (diverse *E. coli*, 30 genomes) and B (O157:H7, 40 genomes), on exclusive nodes of a university cluster (AMD EPYC 7542). Jobs 12739228 and 12739230 finished with no MATCH, audit, or stop failures. An earlier A/hmmsearch run stopped at cached MATCH on genome 5 and is not a speedup. Probe cost in the queued jobs is the singleton schedule, twelve calls at `probe_n = 8`,

$$P = 1595.447 \text{ s.}$$

Stock wall was measured at six positions per collection. Every other genome’s stock wall in the locked total is the screen fit $a + bN_i$, with $a = 132.558 \text{ s}$ and $b = 0.11890 \text{ s/record}$. The locked rule flags a point when $|\text{predicted} - \text{measured}| / \text{measured} > 0.10$. The flag fired at all 12 sampled genomes. On A the measured walls are 384.227 s to 526.950 s, against fitted values 618.958 s to 740.945 s. Relative error runs from 0.351 to 0.772. On B, measured walls are 399.806 s to 506.925 s, relative error 0.457 to 0.807. The fit over-predicts stock time. Imputing the unsampled genomes with that fit inflates cumulative stock wall, and therefore inflates speedup. The fit is not replaced. The locked numbers stand, and the correction is printed beside them.

The correction was computed after seeing the flag. It is post-hoc. Let r be the sum of sampled measured stock walls divided by the sum of sampled fitted stock walls. Sampled stock stays measured. Each unsampled stock wall is multiplied by r . Cached wall and P stay as in the locked analyzer. The assumption is that the sampled ratio represents the unsampled genomes. That assumption is not a measurement of those genomes. r is 0.6277 on A and 0.6053 on B. A fully measured stock series, one wall per genome, is specified and not in the results directory, so it is **[PENDING: per-genome stock completion]**.

Table 5 is the paper pair. The pre-registered prediction at a stand-in $N = 4192$, before these walls, was $2.111 \times$ (A, no P) and $4.077 \times$ (B, no P), or $1.792 \times$ and $3.242 \times$ with the singleton probe. Corrected A lies below both of those predictions. Corrected B without P lies above the no-probe prediction; corrected B with P lies below the with-probe prediction. The protocol’s locked comparison is the imputed total. Both columns are reported because the imputation’s

Table 5: Cumulative hmmsearch wall speedup. Locked: sampled stock measured, other genomes $a + bN_i$. Corrected: post-hoc, unsampled stock scaled by r . “With P ” adds the singleton probe once.

		No P	With P
A	Locked	2.740	2.233
A	Corrected	1.856	1.513
B	Locked	6.847	4.900
B	Corrected	4.404	3.152

Table 6: POST-HOC cumulative hmmscan wall speedup, no probe cost. hmmscan was secondary before the runs. Locked and corrected follow Table 5.

		No P
A	Locked	$3.568 \times$
A	Corrected	$2.95 \times$
B	Locked	$19.468 \times$
B	Corrected	$16.18 \times$

error is measured. Figure 3 plots only measured stock walls, paired with the cached wall at the same genomes, and the four cumulative ratios.

POST-HOC: hmmscan. hmmscan was secondary before the runs. It is not the pre-registered primary. The locked cumulative wall speedups, with no probe cost, are $3.568 \times$ and $19.468 \times$. The same correction, computed after seeing the fit error, lowers them to $2.95 \times$ and $16.18 \times$. Table 6 is that pair. The correction uses the formula in `savings_sensitivity_hmmscan.json`: unsampled stock is the fitted wall times the sampled measured-over-fitted ratio, sampled stock stays measured, and cached wall is unchanged. The assumption, stated there and not measured on the unsampled genomes, is that the sampled ratio represents those genomes.

6.3 RQ3: Reference-side fitter

T1 asks the fitter to recover the hmmscan `-cut_ga` contract with no HMMER normalizer in the code, on the historical R2p and R3c halves (2,435 and 2,436 entries, $Z = 4871$). T1 passes. Both argv bundles ship. Tblout columns 4 and 7 and domtblout columns 6 and 12 are entry count. Domtblout column 11 is per-key row count of the tblout table. Every other numeric column is identity.

On R2p the primary partition left entry count and total entry length tied on columns 4 and 7 (578/578 rows each). The length partition kept entry count and dropped total length (Figure 4). Column 4’s maximum absolute residual is 6.658×10^{-7} under entry count (578/578 consistent) and $2.403 \times$

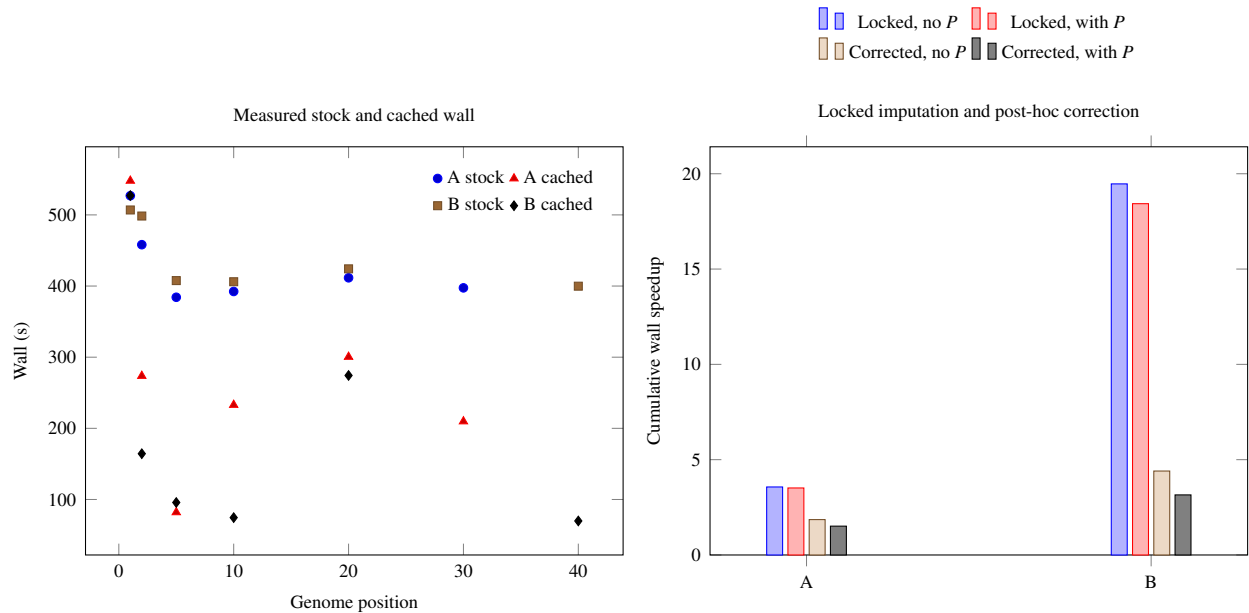


Figure 3: Record-side hmmsearch savings. Left: stock wall only where it was measured (six genomes per collection), with the cached wall at those genomes. Unsamped stock is omitted because the locked total imputes it. Right: cumulative wall speedup. Corrected bars are post-hoc (§6.2).

10^{-5} under total length (17/578). Column 7's maxima are 0.004008 (578/578) and 1.402 (19/578).

One row shows why the two factors separate. On column 4 the worst entry-count row prints $1e-05$ on the part and $1.4e-05$ on the whole, at factor 1.33342. The absolute residual is the column maximum, 6.658×10^{-7} , inside (2). The worst total-length row on the same column prints $3.7e-05$ against $5e-05$ at factor 2.00081, residual 2.403×10^{-5} , which fails the predicate. The primary halves had nearly equal entry counts and nearly equal total length, so both size measures fitted until the length-unbalanced partition pulled the factors apart. Skipping the tie-break would have left those two columns unassigned, and the argv would have refused.

R3c did not tie, so the tie-break did not run. On the primary partition, entry count is consistent on 528/528 tblout rows and 511/511 domain rows of the full-sequence and i-Evalue columns. Total length is 436/528, 449/528, 424/511, and 435/511 on those four columns. Domtblout column 11 fits only per-key row count of tblout (511/511); entry count is 119/511 and total length is 104/511. The earlier domain check that called c-Evalue not reusable is superseded by this assignment: the normalizer is the merged per-sequence row count, which is why the rescale runs after the merge. The kill-test files that predate the fitter already showed the property the fitter had to find. R2p: 1,156/1,156 E-values consistent with entry count and 0 unnormalized mismatches, hit sets equal at 578 hits. R3c: 511/511 on full-sequence E, i-Evalue, and c-Evalue, with equal domain sets. Those files did not compare total entry length and did not run the tie-break. T1

is the comparison.

T2 is the negative control: the same inputs without `-cut_ga`. The union has 1,085 hits and the full run 1,082, with 3 hits only in the union, so the row-key sets differ. The fitter refuses. The recorded reason is that tblout column 16 matches no normalizer, and the primary key sets are unequal. No reuse. A separate 400-model run is underpowered relative to R2p and is not this control.

The synthetic fixtures are not HMMER-shaped. They use the `entryline` format, a column printed to six decimal places and deliberately named `evalue`, seed 20261006, and the pinned shuffle above. Table 7 is the oracle written before the fitter ran. The outcome addendum records every row as passed: A, B, and C ship the member in the table, and D and E refuse. A fitter that skips the tie-break and guesses entry count fails B. A fitter that hard-codes the column name `evalue` fails B the same way. There is no separate results JSON for these fixtures; the record is that outcome addendum. The GeStore baseline refuses law A, which is the synthetic form of a merge that will not rescale.

6.4 RQ4: Pfam 38.1 to 38.2

The pre-registered timing runs five proteomes, three repeats, stock against reference-incremental, on one exclusive node, with ref-merge as the match relation and the count-churn prediction as the gate. The churn file records model-file accounting only and sets timed to false. The speedup is [PEND-ING: measured 38.1→38.2 re-annotation]. Table 3 stays a

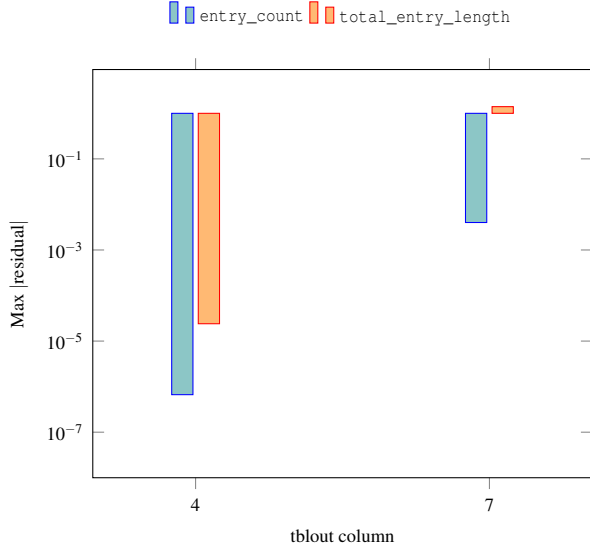


Figure 4: T1 tie-break on R2p tblout. Maximum absolute residual after rescaling. Entry count fits every row. Total entry length does not. Source: `reference_fitter_t1.json`.

Table 7: Pre-registered synthetic oracle. The outcome addendum records each required decision as passed. No HMMER binary is involved.

Law	Required decision
A $ R \cdot p$	ship <code>entry_count</code> , only after the tie-break
B $L(R) \cdot p$	ship <code>total_entry_length</code>
C per-key count $\cdot p$	ship <code>per_key_row_count</code>
D hash rank in this run	REFUSE
E one row per entry pair	REFUSE, key sets differ

projection.

6.5 RQ5: Generality

The second tool, pre-registered before any DIAMOND run, is DIAMOND blastp at a pinned release, compared with iSeqSearch’s hand-built rescale. MMseqs2 is excluded in that protocol because its prefilter couples the hit set to the rest of the database. The comparison is **[PENDING: DIAMOND versus iSeqSearch]**. Smoke artifacts under `results/reference_diamond_smoke/` are not that comparison and are not reported as a speedup or as a fitter decision.

6.6 RQ6: Baselines

Riker, ProcessCache, and INCR key a command or a byte chunk. On a reference release their expected behaviour is a full rerun, because the reference bytes changed. A GeStore-style path that refuses every non-identity numeric column

is implemented and refuses the synthetic entry-count law. A timed comparison on the Pfam release pair, and a head-to-head with INCR on the record-side proteomes, are **[PENDING: Riker, ProcessCache, INCR, and GeStore-style timing]**.

6.7 RQ7: A boundary the fitter refuses

T2 is the pre-registered refusal on hmmscan without `-cut_ga`, and the synthetic rank and pairwise fixtures refuse. A boundary on an unmodified CLI outside that argv pair, where decomposition fails or no closed-family member fits and the result is still a refusal rather than a silent rescale, is **[PENDING: boundary refusal beyond T2 and the synthetic fixtures]**.

7 Related work

Hand-built reference incrementality. iBLAST wraps NCBI BLAST+, builds a delta database from filenames and sizes, and applies a Karlin–Altschul recount or a Spouse rescale $E_{\text{total}} = E_{\text{part}} \cdot n_{\text{total}}/n_{\text{part}}$ that the authors derive [8]. “Automated” means the user does not type the formula. The partition is append-shaped growth of one tool, not a probed partition of an arbitrary reference. iSeqSearch adopts that rescale for tools that emit m8, including MMseqs2 and DIAMOND, and judges E-values by correlation [31]. It does not discover which column is normalized. GeStore runs unmodified binaries, including BLASTP and HMMER, by rewriting inputs and merging outputs through an administrator-written plugin [23]. The BLAST plugin corrects E-values following Turcu, Nestorov, and Foster (DaWaK 2008), as GeStore describes that repair; that PDF was not obtained in the prior-art pass. The HMMER plugin only generates input files, and GeStore’s Pfam accounting marks every update significant. MMseqs2’s `clusterupdate` mode is hand-built incremental clustering, not a search-result cache [29]. InterProScan’s pre-calculated lookup and eggNOG-mapper’s cache mode key the query sequence, not a diff of the member database [5, 12]. The HMMER daemon keeps one database resident for many queries [11]. UniRule recomputes annotations from expert-written rules [19]. Ensembl VEP’s cache is a per-release data dump [21]. None of these infer both decomposition over reference entries and a fitted column normalizer.

Input-side decomposition, and a polynomial search.

KumQuat synthesizes a combiner for an unmodified command so that $f(x_1 ++ x_2) = g(f(x_1), f(x_2))$, in a language of concat, merge, rerun, stitch, add, first, and second [25]. That is decomposition of an input stream. There is no reference argument and no multiplicative database-size term. Caruca classifies opaque commands as stateless, pure, or side-effectful and leaves aggregator synthesis to that line of work [18]. INCR memoizes command effects; finer chunks require crowd-sourced annotations and are keyed by bytes [30]. Zhang et al.

Table 8: Prior-art placement against the two inferences. (a) is decomposition over reference entries. (b) is a fitted numeric normalizer. “Hand-built” means the partition or the formula is written for that tool.

System	(a)	(b)	Code
iBLAST [8]	BLAST, append-shaped	derived E-value formula	hand-built
iSeqSearch [31]	m8 producers	Spouge rescale, copied	hand-built
GeStore [23]	inside a plugin	BLAST plugin only	hand-built
KumQuat [25]	input stream	none	automatic, input-side
Caruca [18]	line statelessness	none	stops before the aggregator
INCR [30]	annotated byte chunks	none	command grain by default
This fitter	probed partition, or refuse	closed family, or refuse	no per-tool branch

infer polynomial metamorphic relations for numeric library functions by particle-swarm search [32]. The relation is between two scalar outputs on affinely related inputs. It does not partition a reference or identify a coefficient with a count of entries. MapReduce and divide-and-conquer synthesis start from examples or a semantic model of a program, not from an opaque CLI and a statistical column [13, 27]. Incremental view maintenance and self-adjusting computation maintain a computation the user has already written down [1, 3].

Record side, which is occupied up to the copy split.

Whole-command caches are the systems above plus workflow task hashes [4, 9, 22]. IncPy and Mandala memoize inside a language runtime [15, 20]. GREYONE infers which input bytes affect which values by mutation [14]. DUST learns which URL components do not change page content [2]. Param Miner finds request inputs that change a response, including inputs reflected verbatim [17]. A cache keyed by any of those dependency sets still contains copied fields. The secondary claim is narrower: transport fields are removed from the key and spliced back. On VCF the dependency recall is 0.4383 and the copy-aware recall is 0.7874, ratio 0.5567, equal to the byte-key ratio because SnpEff copies every field. On GenBank the dependency recall is 0 at every k while copy-aware recall at the last k is 0.4053. The RefSeq control, where accessions are shared, stays at ratio 1.

Three attacks from the prior-art pass. The kill rule was fixed before the search: kill the claim if any prior system infers both (a) and (b) for unmodified black-box tools; narrow it if prior work does one of them automatically for one family, or hand-builds both for a specific tool. The kill branch did not fire. The narrow branch did.

1. *GeStore and iBLAST already did this, including HMMER.* GeStore’s HMMER plugin does not rescale, and iBLAST’s formula is BLAST-specific code. The defeat on disk is that pair of designs together with a fitter that has no per-tool branch. T1 passes on hmmscan and the length fixture ships total entry length on a column named `evalue`. A hand-coded HMMER plugin can be adjusted until T1 passes and still fail the length fixture.

2. *KumQuat already infers decomposition for black boxes.* Its equation splits the input stream, and the combiner language has no reference scale factor. A merge that demands byte equality is what the `no--cut_ga` run rejects: the union is a strict superset of the full run (1,085 vs. 1,082 hits). That combiner would refuse or rerun. It would not fit entry count or per-key row count.

3. *The ASE 2014 search already infers the normalizer, because a rescale is a polynomial.* That search does not partition a reference, choose a table column, or tie the coefficient to a measured size of the reference. The experiment that would make the attack real is fitting the pre-registered family on partition probes of an unmodified CLI. T1 is that experiment for one argv. It is not a result in the ASE paper.

The sentence the verdict leaves standing is the one in the introduction. It does not say that incremental search is new, that E-value correction is new, or that black-box decomposition of an input stream is new.

8 Limitations

F6 is any input the tool reads outside argv: environment variables, unnamed paths, configuration beside the input. The macOS probe run marks both F6-env and F6-file as unsafe-ship (32/32 each). Linux file tracing is the stated cover for F6-file and has no committed measurement here. F6-env remains outside the guarantee. The error bound (3) is a printed-precision relation under a re-print assumption. It is not bit identity of the tool’s internal score, and a column whose law is outside the four members refuses rather than approximating. Parsers are per format. A reference format with no parser refuses; the fitter does not invent a length. All wall-clock numbers in the committed timing files used above were taken on exclusive nodes of a university cluster (AMD EPYC 7542), except the probe-evaluation counts, which are a macOS measurement of decisions rather than of speed. A speedup for the 38.1→38.2 release is not among those numbers.

9 Conclusion

One fitter, with no per-tool code, accepts or refuses a reference-side contract for an unmodified CLI: decomposition over entries, and a normalizer from a four-member family, with a length tie-break. On `hmmScan -cut_ga` it ships entry count and per-key row count; without that flag it refuses. Record-side reuse with a copy-aware key is the secondary result, and its locked speedups are reported next to the post-hoc correction the fit error requires. The release-level speedup, the second search tool, and the whole-command baselines remain to be measured.

Availability

An anonymized artifact accompanies this submission. It contains the fitter and the result files that are the sources of the measured numbers. The repository URL is omitted for double-blind review.

References

- [1] Umut A. Acar, Guy E. Blelloch, and Robert Harper. Selective memoization. In *Proc. ACM SIGPLAN–SIGACT Symposium on Principles of Programming Languages (POPL)*, 2003.
- [2] Ziv Bar-Yossef, Idit Keidar, and Uri Schonfeld. Do not crawl in the DUST: Different URLs with similar text. In *Proc. International Conference on World Wide Web (WWW)*, 2007.
- [3] José A. Blakeley, Per-Åke Larson, and Frank Wm. Tompa. Efficiently updating materialized views. In *Proc. ACM SIGMOD International Conference on Management of Data*, 1986.
- [4] Broad Institute. Cromwell call caching. <https://cromwell.readthedocs.io/en/stable/Configuring/>, 2024. Call caching; documentation.
- [5] Carlos P. Cantalapiedra, Ana Hernández-Plaza, Ivica Letunic, Peer Bork, and Jaime Huerta-Cepas. eggNOG-mapper v2: Functional annotation, orthology assignments, and domain prediction at the metagenomic scale. *Molecular Biology and Evolution*, 38(12):5825–5829, 2021.
- [6] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998. arXiv:2002.12543.
- [7] Charlie Curtsinger and Daniel W. Barowy. Riker: Always-correct and fast incremental builds from simple specifications. In *Proc. USENIX Annual Technical Conference (ATC)*, pages 885–898, 2022.
- [8] Sajal Dash, Sarthok Rasique Rahman, Heather M. Hines, and Wu-chun Feng. iBLAST: Incremental BLAST of new sequences via automated e-value correction. *PLoS ONE*, 16(4):e0249410, 2021.
- [9] Paolo Di Tommaso, Maria Chatzou, Evan W. Floeden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35:316–319, 2017.
- [10] Dune Developers. Dune cache-check-probability. https://dune.readthedocs.io/en/stable/reference/config/cache_check_probability.html, 2024. Documentation.
- [11] Sean R. Eddy. Accelerated profile HMM searches. *PLoS Computational Biology*, 7(10):e1002195, 2011.
- [12] eggNOG-mapper developers. eggNOG-mapper cache mode (`-m cache`). <https://github.com/eggnoget/eggnoget/wiki>, 2021. Product documentation; `-md5` then user-supplied annotations file.
- [13] Azadeh Farzan and Victor Nicolet. Synthesis of divide and conquer parallelism for loops. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017.
- [14] Shuitao Gan, Chao Zhang, Xiaojun Qin, Xuwen Tu, Kang Li, Zhongyu Pei, and Zuoning Chen. GREYONE: Data flow sensitive fuzzing. In *Proc. USENIX Security Symposium*, 2020.
- [15] Philip J. Guo and Dawson Engler. Using automatic persistent memoization to facilitate data analysis scripting. In *Proc. ISSTA*, 2011.
- [16] Samuel Karlin and Stephen F. Altschul. Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proceedings of the National Academy of Sciences*, 87(6):2264–2268, 1990.
- [17] James Kettle. Param miner, 2018. PortSwigger.
- [18] Evangelos Lamprou, Seong-Heon Jung, Mayank Keliya, Lukas Lazarek, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Caruca: Effective and efficient specification mining for opaque software components, 2025. arXiv:2510.14279.

- [19] Alistair MacDougall et al. UniRule: A unified rule resource for automatic annotation in the UniProt knowledgebase. *Bioinformatics*, 2020.
- [20] Alexander Makelov. Mandala: Compositional memoization for simple & powerful scientific data management. In *Proc. Python in Science Conference (SciPy)*, 2024.
- [21] William McLaren, Laurent Gil, Sarah E. Hunt, Harpreet Singh Riat, Graham R. S. Ritchie, Anja Thormann, Paul Flicek, and Fiona Cunningham. The Ensembl variant effect predictor. *Genome Biology*, 17:122, 2016.
- [22] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twardziok, Alexander Kanitz, Andreas Wilm, Manuel Holtgrewe, Sven Rahmann, Sven Nahnsen, and Johannes Köster. Sustainable data analysis with Snake-make. *F1000Research*, 10:33, 2021.
- [23] Edvard Pedersen, Nils Peder Willassen, and Lars Ailo Bongo. Transparent incremental updates for genomics data analysis pipelines. In *Euro-Par 2013: Parallel Processing Workshops*, volume 8374 of *Lecture Notes in Computer Science*, pages 311–320. Springer, 2014.
- [24] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.
- [25] Jiasi Shen, Martin Rinard, and Nikos Vasilakis. Automatic synthesis of parallel Unix commands and pipelines with KumQuat. In *Proc. PPOPP*, 2022.
- [26] Kelly Renee Shiptoski. *Reproducibility and Performance Optimizations for Unmodified Linux Programs*. PhD thesis, University of Pennsylvania, 2023.
- [27] Calvin Smith and Aws Albarghouthi. Mapreduce program synthesis. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2016.
- [28] Sarah Spall, Neil Mitchell, and Sam Tobin-Hochstadt. Build scripts with perfect dependencies. *Proc. ACM Program. Lang.*, 4(OOPSLA), 2020.
- [29] Martin Steinegger and Johannes Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35:1026–1028, 2017.
- [30] Yizheng Xie, Evangelos Lamprou, Jerry Xia, and Nikos Vasilakis. Faster re-execution via bolt-on incrementalization. In *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026.
- [31] Hyunwoo Yoo, Mohammad S. Refahi, Robi Polikar, Bahrad A. Sokhansanj, James R. Brown, and Gail L. Rosen. iSeqSearch: Incremental protein search for iBlast/iMMSeqs2/iDiamond. *PeerJ*, 2025.
- [32] Jie Zhang, Junjie Chen, Dan Hao, Yingfei Xiong, Bing Xie, Lu Zhang, and Hong Mei. Search-based inference of polynomial metamorphic relations. In *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2014.